Friedrich-Alexander-Universität
Erlangen-Nürnberg

# Practical Skills in Computational Biology

## Fundamentals of Bioimage Processing - Unit 2: Image Filtering

**Anna Möller[1, 2]** **Dr. Anne Hartebrodt[1]** **Prof. Dr. Stefan Uderhardt[2]** **Prof. Dr. David B. Blumenthal[1]**

[1]Biomedical Network Science Lab, Department of Artificial Intelligence in Biomedical Engineering, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU),

[2]Department of Medicine 3 – Rheumatology and Immunology, FAU and Universitätsklinikum Erlangen, 91054, Germany

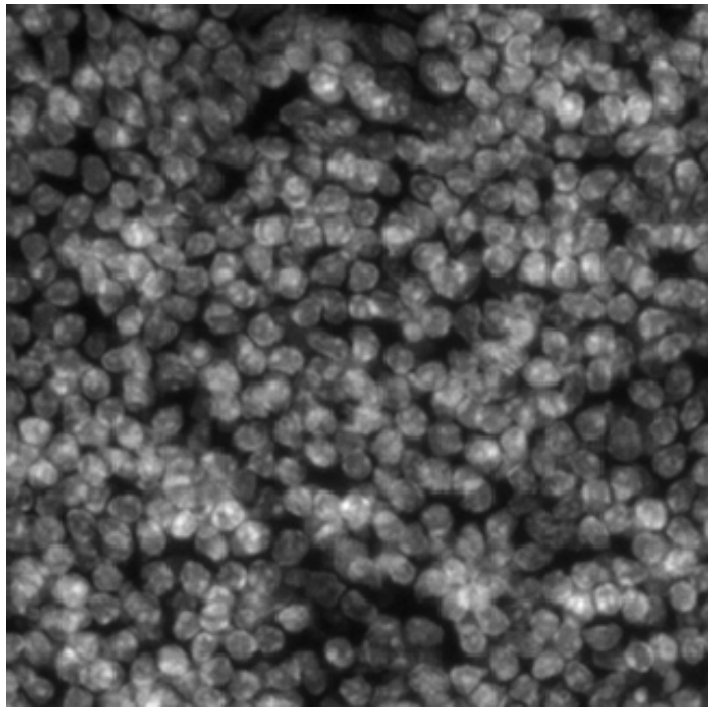
23.06.2026

1. Basic Concepts

2. Common Image Filters

# Image Filtering

- Filtering an image produces a new image.
- The output image is calculated as a function of the neighborhood of the pixels in the input image.

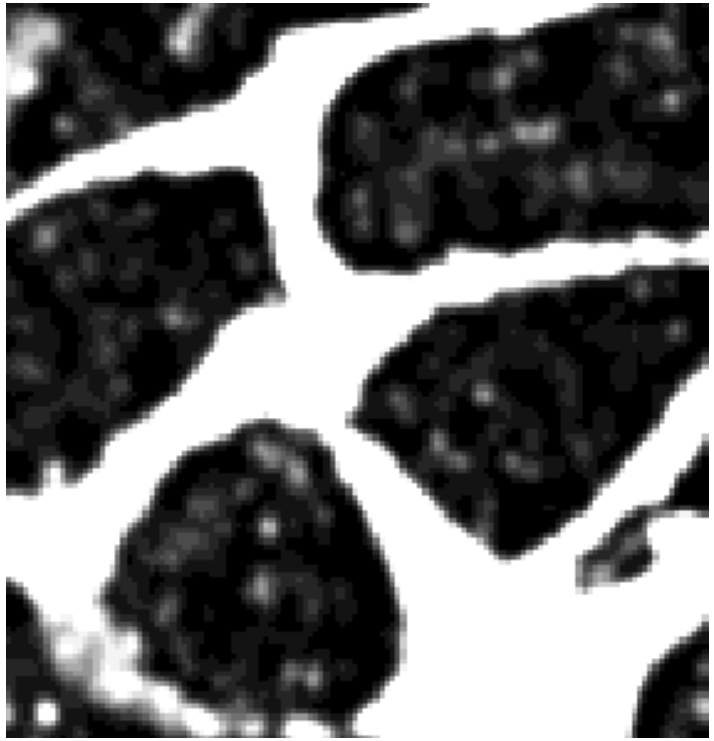


# Filter choice

- Different filters answer different questions through mathematical operations.
- Complex filtering can calculate abstract representations (shape, texture, patterns) of images.
- Application examples:
  - Noise-reduction
  - Artefact-removal
  - Contrast enhancement
  - Correcting uneven illumination

# Image Correction

- Noise: general term for unwanted modifications that can occur during capture, storage, transmission, processing, or conversion of images.
- Extracting numbers from images almost always requires noise removal or image filtering.



# Non-linear Filters

- Linear filters replace each pixel value with a non-linear function of surrounding pixels.
- Example: minimum, maximum, median, variance,...

# Linear Filters

- Linear filters replace each pixel value with a linear combination of surrounding pixels.
- Operation is called convolution.
- Convolutions on images require convolution kernels, small matrices that hold the weights for the weighted addition.

# Definition of 1D Discrete Convolution

The convolution of a discrete input signal $f[n]$ and an impulse response (kernel) $h[n]$ is defined as:

$$(f * h)[n] = \sum_{k=-\infty}^{\infty} f[k]h[n-k]$$

Properties:

- **Linearity:** The operation is linear and shift-invariant.
- **Weighted Average:** Each output $y[n]$ is a weighted sum of the input values, where weights are determined by the kernel.
- **Feature Extraction:** In DSP and CNNs, $h$ acts as a filter to detect specific patterns.

# The Mechanism: Flip and Slide

To compute $(f * h)[n]$ manually, we follow four distinct steps:

1. **Change Variable:** Express both signals in terms of a dummy variable $k$ ($f[k]$ and $h[k]$).
2. **Flip (Reflect):** Time-reverse the kernel to get $h[-k]$. This is required by the minus sign in the formula $h[n-k]$.
3. **Slide (Shift):** Offset the flipped kernel by $n$ units to get $h[n-k]$.
4. **Multiply and Sum:** Multiply overlapping values and sum them up to produce the value at index $n$.
5. If the kernel $h$ is symmetric, flipping has no effect, but the mathematical definition remains the same.

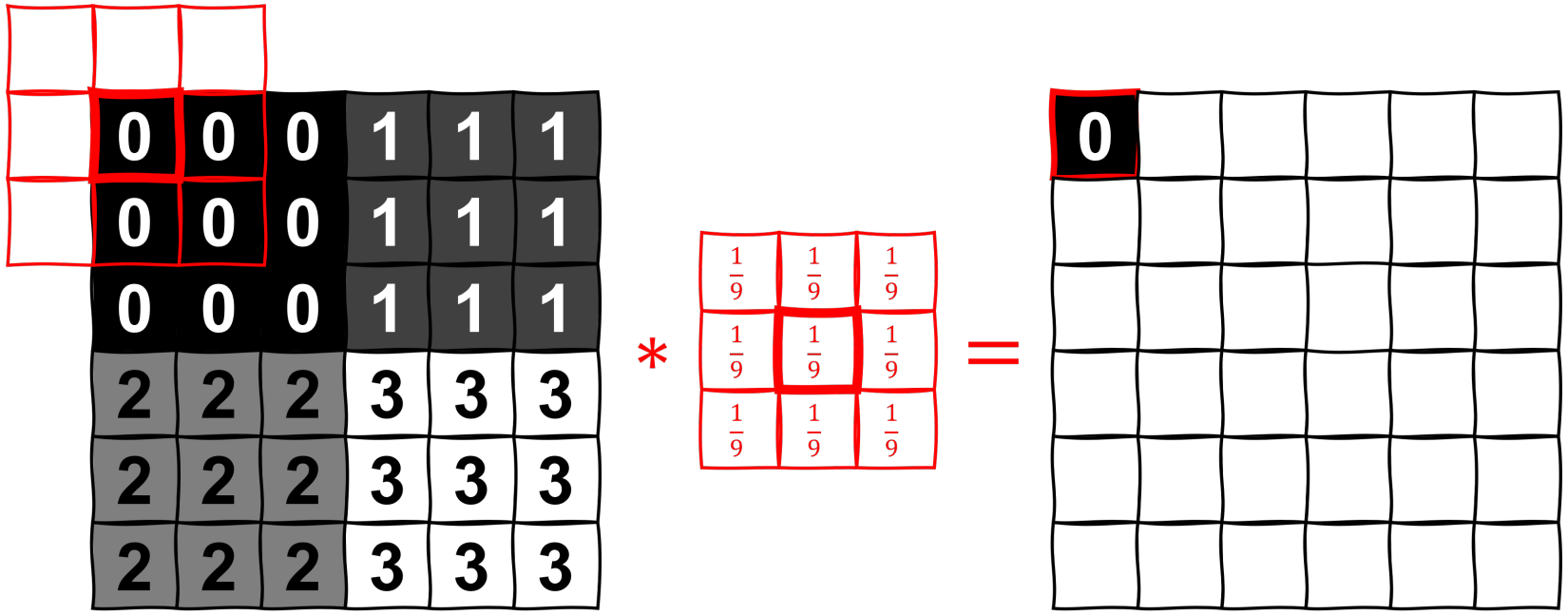
# Definition of 2D Discrete Convolution

The convolution of a 2D input signal (image) $f[m, n]$ and a kernel $h[m, n]$ is defined as:

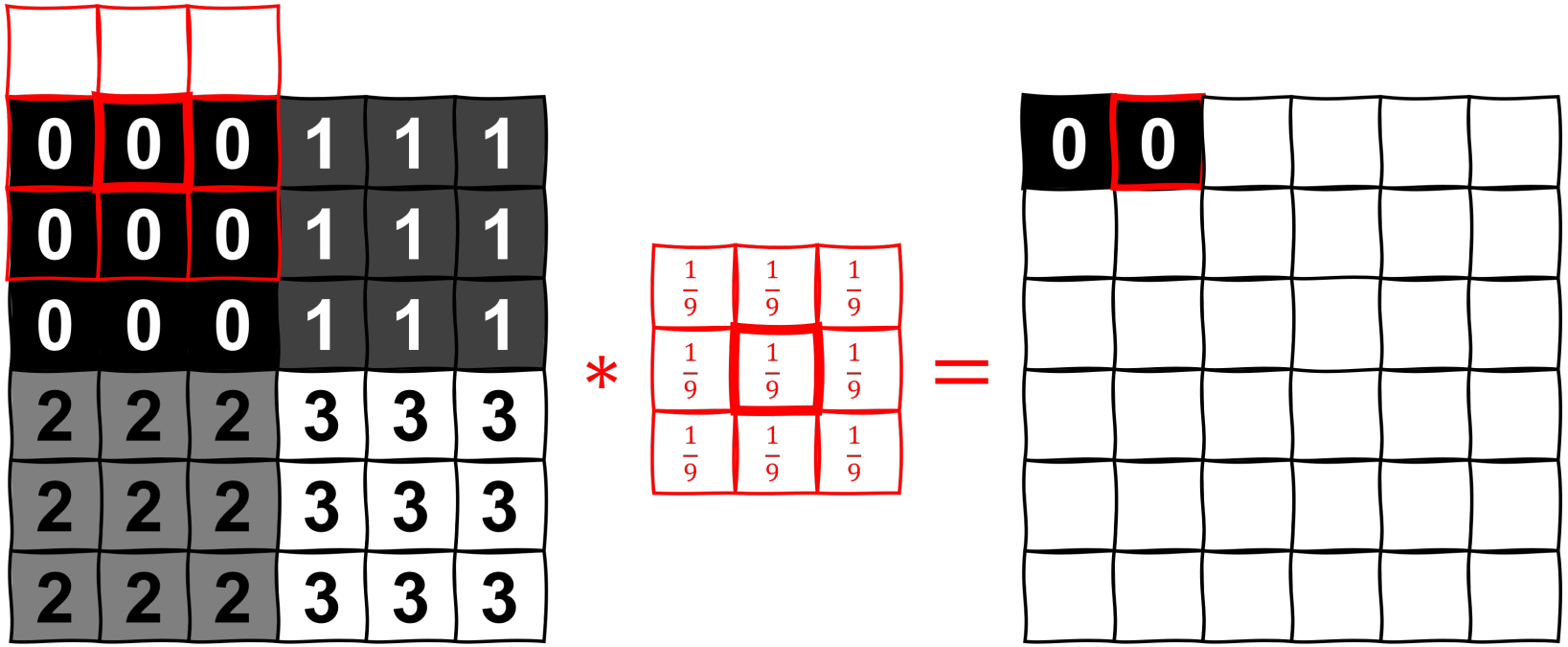
$$(f * h)[m, n] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} f[i, j] \cdot h[m - i, n - j]$$

- **Double Flip:** The kernel $h$ is flipped both horizontally *and* vertically.
- **Spatial Filtering:** The flipped kernel slides across the rows and columns of the input grid.

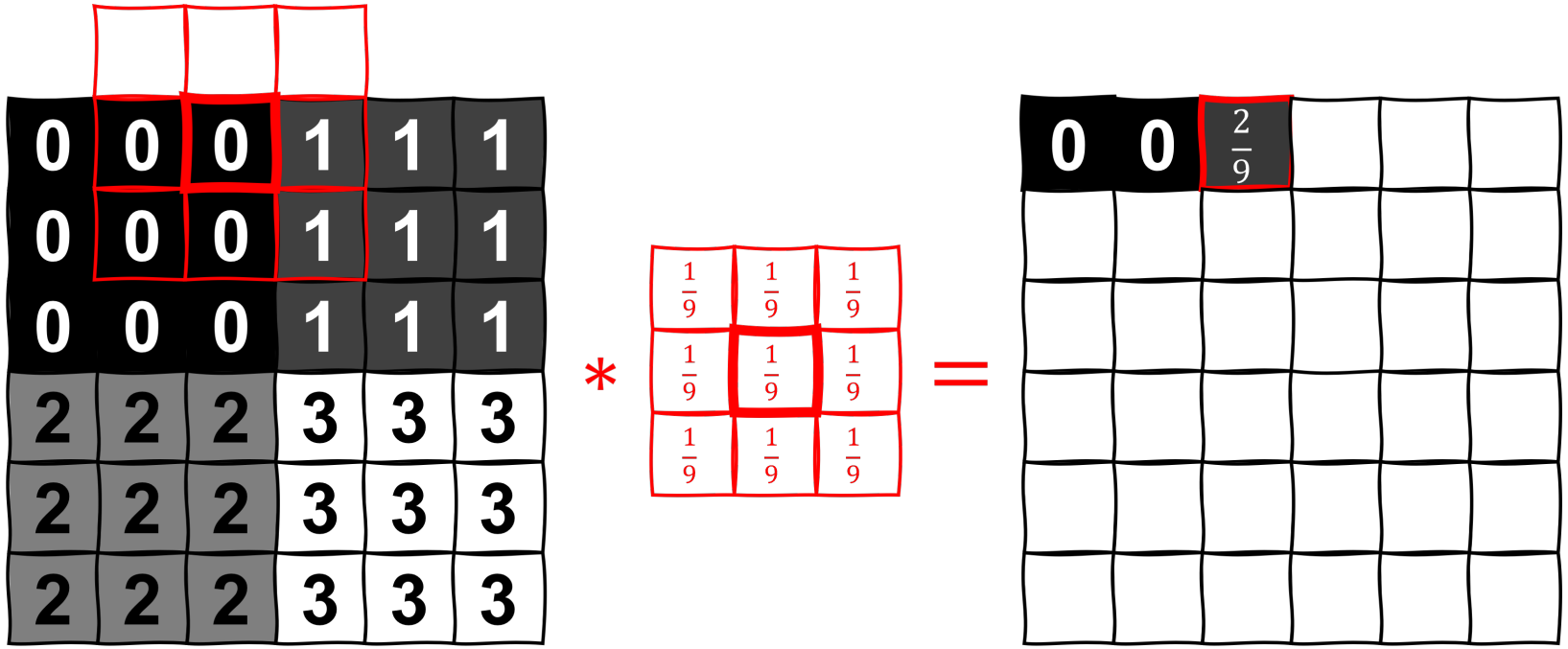
# 2D Discrete Convolution



# 2D Discrete Convolution



# 2D Discrete Convolution



# 2D Discrete Convolution

$$
\begin{bmatrix}
0 & 0 & 0 & 1 & 1 & 1 \\
0 & 0 & 0 & 1 & 1 & 1 \\
0 & 0 & 0 & 1 & 1 & 1 \\
2 & 2 & 2 & 3 & 3 & 3 \\
2 & 2 & 2 & 3 & 3 & 3 \\
2 & 2 & 2 & 3 & 3 & 3
\end{bmatrix}
*
\begin{bmatrix}
\frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\
\frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\
\frac{1}{9} & \frac{1}{9} & \frac{1}{9}
\end{bmatrix}
=
\frac{1}{9} \times
\begin{bmatrix}
0 & 0 & 2 & 4 & 6 & 4 \\
0 & 0 & 3 & 6 & 9 & 6 \\
4 & 6 & 9 & 12 & 15 & 10 \\
8 & 12 & 15 & 18 & 21 & 14 \\
12 & 18 & 21 & 24 & 27 & 18 \\
8 & 12 & 14 & 16 & 18 & 12
\end{bmatrix}
$$

1. Basic Concepts

2. Common Image Filters

# Box Filter: Mean Smoothing

The Box filter is a linear filter where each pixel in the resulting image has a value equal to the average value of its neighbors.

$$y[m,n] = \frac{1}{M^2} \sum_{i=0}^{M} \sum_{j=0}^{M} f[i,j]$$

- **Low-Pass Filter:** Reduces high-frequency components (noise) and blurs sharp edges.

| | | |
|---|---|---|
| $\frac{1}{9}$ | $\frac{1}{9}$ | $\frac{1}{9}$ |
| $\frac{1}{9}$ | $\frac{1}{9}$ | $\frac{1}{9}$ |
| $\frac{1}{9}$ | $\frac{1}{9}$ | $\frac{1}{9}$ |

$3 \times 3$ Averaging Kernel

# Sobel Operator: Gradient Approximation

The Sobel operator calculates the image gradient by convolving with two kernels ($G_x, G_y$). It approximates the partial derivative at each pixel.

- Each kernel performs a **central difference** on the respective axis, approximating the derivative as

$$f'(x) = \lim_{h \to 0} f(x + 0.5h) - f(x - 0.5h)$$

- Simultaneously, each filter applies a 1D Gaussian-like smoothing $[1, 2, 1]^T$ on the orthogonal axis to reduce noise.
- **Magnitude:** The total edge strength is $|\nabla f| = \sqrt{G_x^2 + G_y^2}$.

| | | |
|---|---|---|
| -1 | 0 | 1 |
| -2 | 0 | 2 |
| -1 | 0 | 1 |

$G_x$ (Horizontal)

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 0 | 0 | 0 |
| -1 | -2 | -1 |

$G_y$ (Vertical)

# 2D Gaussian Filtering

A Gaussian filter is a linear smoothing filter used to reduce noise and detail. The 2D discrete kernel is sampled from:

$$G[m,n] = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{m^2+n^2}{2\sigma^2}\right)$$

- **Separability:** $G[m,n] = g[m] \cdot g[n]$. This allows 2D convolution to be performed as two 1D passes (horizontal then vertical).
- **Parameter $\sigma$:** Controls the degree of smoothing. A larger $\sigma$ results in a wider kernel and more blur.

| | | |
|---|---|---|
| $\frac{1}{16}$ | $\frac{2}{16}$ | $\frac{1}{16}$ |
| $\frac{2}{16}$ | $\frac{4}{16}$ | $\frac{2}{16}$ |
| $\frac{1}{16}$ | $\frac{2}{16}$ | $\frac{1}{16}$ |

Example: Normalized integer approximation

# Laplacian Filter: Second-Order Derivative

The Laplacian is an isotropic measure of the 2nd-order spatial derivative. It highlights regions of rapid intensity change and is used for edge detection.

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

- **Zero-Sum:** The kernel coefficients sum to zero, meaning the filter response is zero in areas of constant intensity.
- **Directional Independence:** Being rotationally invariant, it detects edges in all directions equally.
- **Sensitivity:** Highly sensitive to noise; often applied after smoothing.

| | | |
|---|---|---|
| 0 | 1 | 0 |
| 1 | -4 | 1 |
| 0 | 1 | 0 |

Standard $3 \times 3$ Laplacian Kernel

# Laplacian of Gaussian (LoG)

To mitigate the noise sensitivity of the Laplacian, the image is first smoothed with a Gaussian filter. Due to the associativity of convolution:

$$\nabla^2(f * G) = f * (\nabla^2 G)$$

- **Blob Detection:** Excellent for finding structures of a specific scale defined by $\sigma$.

| | | |
|---|---|---|
| -1 | -1 | -1 |
| -1 | 8 | -1 |
| -1 | -1 | -1 |

Discrete LoG Approximation